# Future aware Dynamic Thermal Management in CPU-GPU Embedded Platforms

Srijeeta Maity*, Rudrajyoti Roy*, Anirban Majumder, Soumyajit Dey, Ashish R. Hota
*Indian Institute of Technology, Kharagpur, India*

***Abstract*—Modern data intensive Cyber-physical Systems ubiquitously employ heterogeneous multiprocessor systems-on chips (MPSoCs) for real-time sensing, computation, and actuation. The low foot-print of such SoCs often leads to high operating temperatures beyond acceptable limits. In this context, conventional thermal management techniques such as Operating System (OS) governed frequency scaling result in drastic degradation of the quality of experience and violation of real-time requirements. In this work, we propose an analytical thermal model for heterogeneous CPU-GPU embedded platforms and demonstrate a Model Predictive Control (MPC) based scheduling strategy with a novel heuristics-based optimization technique that leverages information about future kernels to judiciously choose suitable task mapping options for minimization of the platform's peak (or maximum) temperature to prolong chip's life span while adhering to real-time performance requirements. To the best of our knowledge, this is the first work that considers future awareness along with a variety of online task mapping control actions such as partitioning, migration, and frequency tuning in the context of thermal management in heterogeneous CPU-GPU embedded platforms. We evaluate the proposed heterogeneous framework on an Odroid-XU4 board using OpenCL based workloads and demonstrate its effectiveness in reducing the platform peak temperature.**

***Index Terms*—Heterogeneous Computing, MPC, Adaptive Thermal Management**

## I. Introduction

With advancements in silicon technologies, modern cyber-physical systems (CPSs) are enabled with high performance real-time compute primitives. Examples include Advanced Driver Assistance Systems (ADAS), now ubiquitous in the high volume passenger vehicle segment, and consumer devices such as smartphones. They come equipped with heterogeneous multi-cores comprising both high-performance and low-power cores along with Graphics Processing Units (GPUs), and specialized Neural Processing Units (NPUs). Such data crunching capabilities result in significant power dissipation within a small form-factor. As a result, thermal management has become critical to avoid unfavourable scenarios such as performance degradation and sudden device restarting. For example, recently Tesla recalled 130,000 vehicles for CPU overheating problems that caused the center screen display to reboot during supercharging [1]. Hardware based on-platform cooling solutions such as fans, heat sinks, liquid cooling, etc. are often not suitable for mobile embedded devices due to space constraints. Thus, managing the on-chip temperature while meeting application timing constraints remains a key system design objective that has gotten further challenging in modern heterogeneous embedded platforms due to 1) platform complexity and heterogeneity, 2) increased bandwidth demand from embedded workloads (e.g. neural network pipelines processing high resolution frames at high frequency), and 3) dynamic nature of resident periodic tasks, i.e. scenario based changes in task periods and input size.

Dynamic Thermal Management (DTM) refers to a range of possible hardware and software strategies that work dynamically at run-time to control a chip's operating temperature. When the temperature of any Processing Element (PE) exceeds some predefined threshold, the power consumption of the PE is forcibly reduced by DTM techniques [2]. Common DTM methods include processor level techniques such as clock or fetch gating [3], and software-directed Dynamic Frequency/-Voltage Scaling (DVFS) [4], [5], [6], [7]. However, for such DTM techniques, the focus remains solely on reducing the chip temperature which can affect the running applications' performance resulting in degraded Quality of Service (QoS) and missed deadlines. This is also true for other high-level DTM techniques including temperature-oriented task migration approaches proposed for many-core systems [8], [9]. For latency-sensitive applications such as real-time tasks, latency-unaware DTM can thus be fatal. Thus, there is a critical need for resource managers that allocate compute resources with suitably tuned task mapping configurations such that both latency and thermal constraints are maintained.

There is an increasing demand for supporting dynamic task scheduling primitives in adaptive software systems for varying workload distributions. This essentially requires dynamically accommodating new instances of tasks with reduced periodicity (i.e., job injection) in the current schedule [10]. For example, ADAS applications including pedestrian/traffic-sign detection, lane departure warning, etc. can have scenario-based modifications in sampling rates for better control performance and co-scheduling requirements [11], [12]. Thus, dynamic modification in executing job sets demands adaptive resource management strategies for satisfying both thermal and execution time constraints at runtime.

Existing works on thermal-aware adaptive resource management focus on maintaining an on-chip thermal envelope while maximizing the performance in terms of latency or throughput [13]. They adaptively modify the mapping configurations for the upcoming task based on the information from the previous time step or previous iteration of the task without considering future task behaviour [14]. Such adaptive thermal management schemes, restrict the peak temperature below a tolerable threshold. Even though the thermal constraint is

The authors acknowledge generous support and grant from *Intel India Research Fellowship Program 2021*. *These authors contributed equally.

2022 IEEE Real-Time Systems Symposium (RTSS) | 978-1-6654-5346-2/22/$31.00 ©2022 IEEE | DOI: 10.1109/RTSS55097.2022.00041

2576-3172/22/$31.00 ©2022 IEEE
DOI 10.1109/RTSS55097.2022.00041



Authorized licensed use limited to: Wuhan University. Downloaded on September 20,2026 at 11:58:48 UTC from IEEE Xplore. Restrictions apply.

satisfied, there is scope to reduce the peak temperature further without hampering real-time constraints. Such approaches also do not focus on maintaining a smooth thermal profile which delays in-circuit ageing [15] that helps in prolonging both battery and chip life span. Abrupt thermal spikes due to a sudden increase in resource requirements can be reduced by considering future thermal behaviour, while scheduling the dynamic job set. Thus, there is a need for adaptive, future aware scheduling mechanisms that can perform dynamic mapping by online resource analysis and re-configure scheduling decisions to minimize peak temperature while satisfying real-time constraints in heterogeneous CPU-GPU platforms.

In this paper, we propose a thermal-aware scheduling framework that aims to achieve smoother control of scheduling actions that accounts for future tasks' behavior, to minimize the overall peak operating temperature of the platform while satisfying the deadline constraint for task sets that can vary dynamically. We can solve this optimization problem offline using powerful constraint solvers but such static solutions will fail to scale for dynamic scheduling and can be overly pessimistic [16]. There do exist a few embedded constraint solvers for solving the optimisation problem online but converting the problem to a standard form that is taken as input by these solvers is non-trivial [17]. We solve this discrete-time optimal control problem using a well known control-theoretic approach named *model-predictive control* (MPC) [18]. However, the implementation overhead of classical MPC makes it unsuitable for real-time tasks. So, we propose a novel heuristic algorithm based on beam search and pruning for solving the underlying optimization problem in polynomial time.

The major contributions of our work are summarised as-

1) An analytical thermal model for CPU-GPU heterogeneous platforms to predict the maximum platform temperature during the execution of a given task has been developed.
2) A novel approximate heuristic for MPC based real-time scheduling has been proposed.
3) An OpenCL based scheduling framework has been implemented for this future aware dynamic thermal management technique.
4) The efficiency of the proposed framework has been established on a real embedded CPU-GPU platform through extensive experimentation.

## II. Background and Motivation

### A. Background

***Heterogeneous Programming Model*** *:* Modern embedded computing platforms have seen a paradigm shift in their architectures and have become increasingly heterogeneous with multiple processing elements, such as big-LITTLE CPUs, and GPUs integrated on the same die with a shared memory unit. Instead of using fixed (i.e., static) roles for each kind of device (e.g., CPUs for sequential and management tasks and GPUs for parallel work), heterogeneous programming models, such as OpenCL (by Khronos), oneAPI (by Intel), and HetCompute (by Qualcomm), allow for efficient execution of a single data parallel kernel using more than one of the available devices. We have implemented our framework in OpenCL (Open Computing Language) [19] which provides a standard interface for parallel computing on different compute devices such as CPU, GPU, FPGA, and DSP. An OpenCL program typically consists of two distinct entities: (i) the *host* code which orchestrates the entire process of data transfer and issuing directives for parallel execution and (ii) *kernel* code which executes the actual data parallel computation on compute devices. The smallest processing element of a kernel instance, called work-item operates on a single data point. A group of work-items forms a work-group, and these items execute concurrently on the processing elements of a single compute unit.

***OpenCL Task partitioning*** *:* Partitioning tasks in the context of OpenCL applications entail splitting the data space to be processed by the task/kernel into segments and processing the segments concurrently using the devices of the heterogeneous system. We note that partitioning OpenCL tasks only require configuring the kernel launch commands from the host program and do not require modifying the source code of the kernels. The overhead of partitioning is thus in issuing extra kernel launch directives, which is negligible when compared to the overall execution time of the kernel. In contrast to the traditional approach of executing tasks on a single accelerator device in its entirety, partitioning allows for optimum resource utilization of the different processing elements in a heterogeneous platform, leading to reduced execution times.

***Model Predictive Control*** *:* MPC [18] is an advanced method of process control that explicitly employs a model to solve an optimal control problem over a finite sequence of control actions at each sampling instance. MPC relies on dynamic models of the system, most often linear empirical models obtained by system identification.

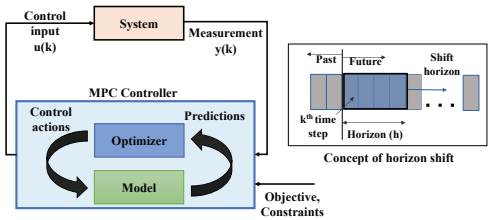



Figure 1: Schematic diagram of MPC and horizon shifting

In Fig. 1, the MPC instance is illustrated schematically. The controller consists of the system (plant) model which maps the set of present state and input variables to the corresponding next state and output variables. At each time step $k$, an MPC controller first measures the current state of the system $x(k)$ to determine which control input $u(k)$ to provide by finding the actions that, over a prediction horizon of $h$, give the best predicted performance by minimizing a given dynamic cost function subject to operational constraints (carried out by the optimizer). The control variables for the first prediction step are applied to the system. The system then transitions to a new state, $x(k+1)$, after which the cycle repeats.



Authorized licensed use limited to: Wuhan University. Downloaded on September 20,2026 at 11:58:48 UTC from IEEE Xplore. Restrictions apply.

### B. Motivating Application

In Advanced Driver Assistance Systems (ADAS), image identification, segmentation, and object detection pipelines such as YOLOlite [20] and LeNet [21] are extensively used. These pipelines are composed of fixed kernel types like Convolution, Pooling, Fully connected and SoftMax kernels in a specific order and dimension. Even though the kernels are known in advance, their periodicity can be dynamic, depending on the quality of control as dictated by environmental requirements. So, it necessitates dynamic resource allocations unless a worst case resource-hungry schedule is always assumed.

Let us consider the scheduling scenario depicted in Figure 2 to elaborate on the advantages of future aware dynamic thermal management. We consider a Directed Acyclic Graph (DAG) consisting of three different types of OpenCL kernels- *Pooling* (memory intensive), *2D Matrix Multiplication* (very compute intensive), and *2D Convolution* (medium compute intensive). Each OpenCL kernel is of dimension $1024 \times 1024$ and is non-preemptive in nature. To keep the experiment simple, we have considered that all kernels are mapped to the CPU and frequency tuning is the only mapping decision. The deadline of the DAG is assumed as 3 sec. marked by the dotted black line in the figure. The experiment is conducted on the target platform Odroid XU4. When aggressive performance-boosting (2 GHz.) for all kernels is applied choosing the maximum frequency setting, the deadline constraint is satisfied, but it reaches trip point (90 °C) and triggers thermal throttling during scheduling. Whereas, when reduced frequency settings (600 MHz.) for all kernels are applied for aggressive thermal management, it results in deadline violation.

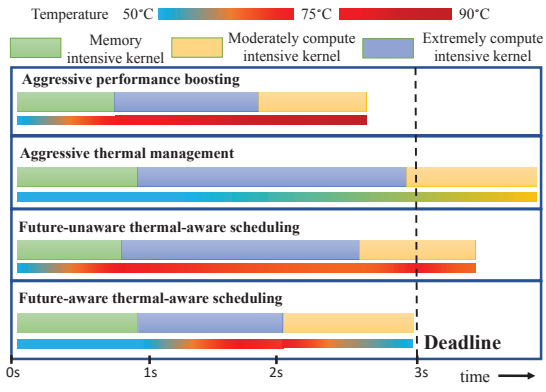



Figure 2: Motivating Application

In the case of thermal-aware scheduling [14] where future kernel patterns are not known, feedback based scheduler takes frequency mapping decisions based on past performance to meet target throughput without hampering the thermal envelope. For the first memory intensive kernel, the scheduler tries to allocate an aggressive frequency setting (2 GHz.) having no past data to meet the target throughput. However, that leads to high device temperature (86 °C). Then the scheduler chooses a lower frequency setting (600 MHz) for the next compute intensive kernel leading to reduced temperature (60 °C) but increased latency. The platform temperature (being a function of start device temperature and utilisation) did not drop further even at a lower frequency setting because the initial device temperature for the second kernel was high and the kernel executed for a longer time. For the last kernel, the scheduler had to choose a moderate frequency setting (1200 MHz) to avoid thermal throttling even at the cost of the deadline miss.

In case of our proposed future-aware and thermal-aware scheduling the future kernel patterns are known, so the scheduler can solve an optimization problem considering all three kernels. It observes that the highly compute intensive second kernel can be leveraged to extract very high throughput if executed at moderately high frequency (1500 Mhz.), but might lead to thermal throttling if initially the platform is not sufficiently cool. Therefore, it schedules the initial memory intensive kernel at the lowest frequency setting (300 Mhz.), thereby keeping the platform cool. Finally, the third kernel is scheduled at a moderate frequency (900 Mhz.) to effectively cool the platform while not violating the deadline, as shown in Figure 2. The peak temperature reached is much lesser (77 °C).

## III. System Model and Problem Formulation

***Platform model*** *:* A general heterogeneous integrated processing platform denoted by $\mathcal{D}$ consists of multiple devices (processing elements) with different compute capabilities. Each device $\delta_i \in \mathcal{D}$ is potentially distinct from another device $\delta_j \in \mathcal{D}, i \neq j$, in terms of the architecture, computational power, memory structure and thermal behaviour. The devices reside on the same die and share a common DRAM. Without loss of generality, we have considered one CPU and one GPU device on our platform, i.e., $\mathcal{D} = \{\delta_C, \delta_G\}$, where $\delta_C$ and $\delta_G$ denote the CPU and the GPU devices, respectively. The voltage at each frequency level is automatically set by the hardware itself. Hence, by DVFS we can only tune the frequency setting. The platform supports a set of discrete operating clock frequencies for each compute device denoted by $F_C$ and $F_G$ for $\delta_C$ and $\delta_G$ respectively. In our work, the value of trip point for both $\delta_C$ and $\delta_G$ are the same. This tripping temperature is denoted by $T_{trip}$. If the temperature goes beyond $T_{trip}$, the frequency setting of the hotter device is reduced drastically by the OS (*thermal throttling*) to reduce temperature, and this can lead to potential deadline misses.

***Task model*** *:* We consider a set of $M$ periodic OpenCL applications for scheduling denoted by $\mathcal{A} = \{\mathcal{A}_1, \mathcal{A}_2, \cdots \mathcal{A}_M\}$. Each application $\mathcal{A}_x$ is represented by the quadruple ($\pi_x$, $D_x$, $a_x$, $\mathcal{K}_x$), where $\pi_x$, $D_x$ and $a_x$ are the period, relative deadline, and arrival time of the application. $\mathcal{K}_x$ denotes a set of $N_x$ data-parallel OpenCL kernels arranged in a Directed Acyclic Graph (DAG) in an application. If we restrict to DAGs, in the form of totally ordered chains, the $y^{th}$ OpenCL kernel in dag $\mathcal{K}_x$ is denoted by $\kappa_{x,y}$. We have used the terms OpenCL kernel and task interchangeably in the rest of this paper. The relative deadline of $\kappa_{x,y}$ is set as $(D_x - \sum_{i=y+1}^{N_x} l_{x,i})$ where $l_{x,y}$ is the Worst Case Execution Time (WCET) of $\kappa_{x,y}$ (profiled offline). This is true of applications (CNN pipelines) considered in the current work. In general, the formulation can be easily modified.



Authorized licensed use limited to: Wuhan University. Downloaded on September 20,2026 at 11:58:48 UTC from IEEE Xplore. Restrictions apply.

Each OpenCL kernel $\kappa_{x,y}$ is characterised by $W_{x,y}$, the number of OpenCL work-groups and $w_{x,y}$, the number of work-items per work-group to be launched. Kernels can be partitioned into suitable smaller sized *sub-tasks* to distribute the workload across multiple devices. For a kernel $\kappa_{x,y}$ in platform $\mathcal{D}$, we use the following partitioning ratio set given by $p_{x,y} \in P = \{0, 20, 40, 60, 80, 100\}$. Mapping the kernel with $p_{x,y}$ indicates $W_{x,y} \cdot (p_{x,y}/100)$ work-groups will be assigned to $\delta_C$ and the remaining work-groups will be assigned to $\delta_G$. Using such partitioning is a unique feature of our framework.

We define each task mapping configuration or simply mapping configuration $s \in \mathcal{S}$ as a tuple of $(p \in P, f \in \mathcal{F})$ where $f$ is the DVFS (or frequency) settings and $\mathcal{F} = F_C \times F_G$ denotes the set of all DVFS configurations of the platform. $\mathcal{S}$ is the set of all possible mapping configurations in which a kernel can be scheduled in platform $\mathcal{D}$. It is the cross product of all partitioning ratios ($|\mathcal{P}|$) and all the discrete processor frequency settings for the CPU ($|F_C|$) and GPU ($|F_G|$) devices in the platform.

The hyper-period ($H$) of the periodic application set $\mathcal{A}$ is given by $l.c.m.(\pi_1, \cdots, \pi_M)$. The behaviour pattern in each hyper-period repeats infinitely until there is any dynamic change in the periodicity of any application. In a given hyper-period $H$, each application $\mathcal{A}_x$ will have $(H/\pi_x)$ instances. If $\pi_x$ changes, the number of instances in the hyper-period will also change. For a given hyper-period $H$, we consider a set of OpenCL kernels $\mathcal{K}^H = \{\kappa^i_{x,y} |\ x \in [1, M]\ ,\ y \in [1, N_x]$ and $i \in [1, H/\pi_x]\}$ that contains all the instances of all the OpenCL kernels in all applications in $\mathcal{A}$ in the hyper-period $H$. We denote each OpenCL kernel in $\mathcal{K}^H$ by $\kappa_k$ where $k \in [1, |\mathcal{K}^H|]$. Unless any dynamic changes modify the periodicity of any application in $\mathcal{A}$, $\mathcal{K}^H$ will be uniform for subsequent hyper-periods. In case of changes in the periodicity of any application, the hyper-period $H$ changes to some $H'$, along with the change in job set from $\mathcal{K}^H$ to $\mathcal{K}^{H'}$. The change is admitted at the end of current hyper-period $H$, allowing all the kernels in $\mathcal{K}^H$ to finish their execution. The modified *job set* $\mathcal{K}^{H'}$ is assumed to be generated such that the order of the kernels follows Earliest Deadline First scheduling policy.

***Overall Optimisation Problem*** *:* For $\mathcal{K}^{H'}$, the mapping configuration for each kernel needs to be updated. For each kernel $\kappa_k \in \mathcal{K}^{H'}$, let the peak or the maximum temperature of the platform $\mathcal{D}$ during execution with the mapping configuration $s$ be denoted by $\mathcal{T}_k(s)$, the corresponding worst-case latency be denoted by $l_k(s)$ and the relative deadline be $d_k$. Then, for thermal aware scheduling, the overall optimization problem over hyper-period $H'$ for the job set $\mathcal{K}^{H'}$ is given by,

$$\min_{s \in \mathcal{S}} \max_{k \in [1, |\mathcal{K}^{H'}|]} \mathcal{T}_k(s) \text{ such that } l_k(s) \leq d_k \ \forall k \in [1, |\mathcal{K}^{H'}|].$$

Considering a MPC based technique with a suitable choice of horizon length ($h \leq |\mathcal{K}^{H'}|$) that recedes till the end of $H'$, we have the following optimization problem for each kernel $\kappa_c$,

$$\min_{s_k \in \mathcal{S}} \max_{k \in [c+1, c+h]} \mathcal{T}_k(s_k) \text{ such that } l_k(s_k) \leq d_k \ \forall k \in [c+1, c+h] \quad (1)$$

The solution of Eq. 1 is a set $\{s_i \,|\, i \in [c+1, c+h]\}$. For static optimization, the optimizer is executed only once whenever any periodicity change occurs (with $h = |\mathcal{K}^{H'}|$) and all $s_i$ thus obtained, are statically mapped to $\kappa_i$. However, for dynamic optimization, only $s_{c+1}$ will be actually mapped to the upcoming kernel $\kappa_{c+1}$ before the window shifts to $[c+2, c+h+1]$.

## IV. Methodology

The proposed framework is designed using multiple hierarchical controllers, as depicted in Fig. 3. The top-level controller uses future aware MPC and is invoked whenever the relative ordering between different kernels in $\mathcal{K}^H$ in a hyper-period gets modified to $\mathcal{K}^{H'}$ due to a change in their task periodicity. The MPC controller runs for each kernel in $\mathcal{K}^{H'}$ and then remains inactive for subsequent hyper-periods until the task set changes.

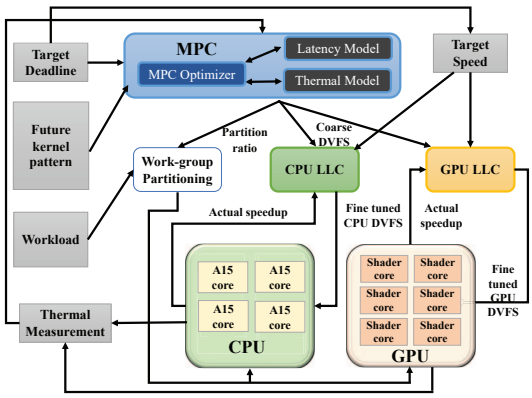



Figure 3: Hierarchical Architecture of the framework

For each kernel in $\mathcal{K}^{H'}$, the MPC takes the pattern of upcoming $h$ (MPC horizon length) kernels in the hyper-period and their respective target deadlines. The latency and the platform peak temperature during the execution of an upcoming kernel are predicted using a thermal and latency model respectively. The actual latency and temperature readings measured from each device in the platform at the end of each kernel execution are also shared as feedback to the MPC controller. The output of the MPC optimizer for each kernel is the task partition ratio $p$ for kernel partitioning across $\delta_C$ and $\delta_G$ and their respective DVFS configuration $f$.

To reduce the search space for the optimization problem in MPC and minimize the scheduling overhead, the MPC controller explores coarse frequency divisions and kernel partitioning among CPU and GPU. In coarse frequency divisions, instead of considering all available discrete frequency settings, only four frequency values $(lo, mid_1, mid_2, hi)$ are considered for device $\delta_C$ and two frequency values $(lo, hi)$ for device $\delta_G$, thus clubbing multiple frequency settings together.

The device frequency setting is further fine-tuned using light weight *Low Level Controllers* or LLCs. LLCs execute for all kernels in all hyper-periods and take care of any deviation from the predicted computation and actual computation measure of each kernel in terms of speedup by fine-tuning the CPU



Authorized licensed use limited to: Wuhan University. Downloaded on September 20,2026 at 11:58:48 UTC from IEEE Xplore. Restrictions apply.

and GPU frequencies in real-time to meet a target speedup computed from the task deadlines. As described in Fig. 3, there are two LLCs, each for $\delta_C$ and $\delta_G$ and they function in a closed loop, feedback control mechanism that returns discrete frequency settings for their respective devices. Before we elaborate on each of these components- MPC models, MPC optimizer, and LLC in the following subsections, we summarise all the symbols used in the paper in the Table I.

| Symbol | Description |
|---|---|
| $i$, $j$, $k$, $x$, $y$ | Iterator or index of an element in a set. |
| $\mathcal{D}$ | Target platform consisting of multiple compute devices. |
| $\delta_i$, $\delta_j$ | $i^{th}$ and $j^{th}$ compute devices in $\mathcal{D}$. |
| $\delta_C$, $\delta_G$ | CPU and GPU compute devices in $\mathcal{D}$ respectively. |
| $F_C$, $F_G$ | Set of discrete operating frequencies in $\delta_C$ and $\delta_G$ respectively. |
| $\mathcal{F}$ | $F_C \times F_G$. |
| $P$ | Set of all possible partitioning ratio. |
| $T_{trip}$ | Tripping temperature beyond which OS induced thermal throttling is triggered in $\mathcal{D}$. |
| $\mathcal{A}$ | Set of considered OpenCL applications to be scheduled. |
| $M$ | Number of applications in $\mathcal{A}$ . |
| $\mathcal{A}_x$ | $x^{th}$ OpenCL application. |
| $\pi_x$, $D_x$, $a_x$ | Period, relative deadline and arrival time of the application $\mathcal{A}_x$. |
| $\mathcal{K}_x$ | Set of data-parallel OpenCL kernels arranged in a Directed Acyclic Graph (DAG) in application $\mathcal{A}_x$. |
| $N_x$ | Number of OpenCL kernels in $\mathcal{K}_x$ . |
| $\kappa_{x,y}$ | $y^{th}$ OpenCL kernel in dag $\mathcal{K}_x$ of application $\mathcal{A}_x$. |
| $W_{x,y}$ | Number of OpenCL work-groups in kernel $\kappa_{x,y}$. |
| $w_{x,y}$ | Number of work-items per work-group in kernel $\kappa_{x,y}$. |
| $p_{x,y}$ | Partitioning ratio of kernel $\kappa_{x,y}$. |
| $H$ | Hyper-period of the periodic application set $\mathcal{A}$. |
| $\mathcal{K}^H$ | Set of all OpenCL kernels of all instances of all applications in hyper-period $H$. |
| $\kappa_k$ | $k^{th}$ kernel in set $\mathcal{K}^H$. |
| $f$ | Frequency (or DVFS) configuration ($\in \mathcal{F}$) consisting of a CPU Frequency $f_C (\in F_C)$ and GPU Frequency $f_G (\in F_G)$. |
| $s$ | Task mapping configuration which is tuple of ($p \in P$, $f \in \mathcal{F}$). |
| $S$ | Set of all possible task mapping configurations. |
| $l_k(s)$ | WCET of kernel $\kappa_k$ executing with mapping configuration $s$. |
| $d_k$ | Relative deadline of kernel $\kappa_k$. |
| $C_i$ | Thermal capacitance of device $\delta_i$ |
| $R_{ij}$ | Thermal cross-resistance between devices $\delta_i$ and $\delta_j$. |
| $R_{ii}$ | Thermal self-resistance of device $\delta_i$. |
| $\psi_i^f$, $\phi_i^f$ | Temperature dependent and independent power coefficients in thermal model for device $\delta_i$ running at frequency $f$. |
| $A_f$, $B_f$ | Trainable coefficient matrices for thermal model for frequency $f$. |
| $PD_i^f(t)$ | Instantaneous power dissipation by device $\delta_i$ running at frequency $f$ at time $t$. |
| $\theta_i^f(t)$ | Temperature of device $\delta_i$ running at frequency $f$ at time $t$. |
| $\theta_i^f(t_\infty)$ | Asymptotic convergence temperature of device $\delta_i$ with constant maximum device utilization at frequency $f$. |
| $\dot{\theta}_i^f(t)$ | Time rate of change of temperature of device $\delta_i$ running at frequency $f$ at time $t$. |
| $\theta_A$ | Surrounding Ambient Temperature. |
| $\Theta^f(t)$ | Vector containing all device temperatures $\theta_i^f(t)$, $\forall \delta_i \in \mathcal{D}$ at frequency $f$. |
| $\mathcal{T}_k(s)$ | Peak temperature of platform $\mathcal{D}$ during the execution of $\kappa_k$ with configuration $s$. |

Table I: List of used symbols

### A. MPC Model Design

For the MPC, we employ predictive models characterizing the timing behaviour and thermal dynamics of the system.

*1) **Latency Model** :* For the latency model of the CPU and GPU for OpenCL tasks, we adopt a simple profile-based model with negligible overhead. We maintain a lookup table for the latency model where each kernel ($\kappa_k$) and each configuration $s$ together constitute the key, and the value corresponding to that key is a tuple of latencies ($l_k^C(s)$,$l_k^G(s)$) where $l_k^C(s)$ and $l_k^G(s)$ are the expected latencies of $\kappa_k$ executed with mapping configuration $s$ in $\delta_C$ and $\delta_G$ respectively. Each OpenCL kernel is profiled 100 times for each mapping configuration in an offline stage and the average of the 100 iterations is populated in the lookup table as the expected latencies. During online scheduling, the latency ($l_k(s) = \max(l_k^C(s), l_k^G(s))$) for any OpenCL kernel $\kappa_k \in \mathcal{K}^H$ executing in a specific mapping configuration $s$ can be instantly accessed.

*2) **Analytical Thermal Model** :* The considered system model supports scheduling at the granularity of the device (not core). For thermal modelling, each device $\delta_i$ can be modelled as an RC circuit [22], where R and C are the thermal resistance and capacitance of the device. In the case of a multi-core CPU-GPU platform integrated on a single die, thermal coupling between multiple devices results in a more complex thermal model given by [22]. The expression is given by,

$$C_i\dot{\theta}_i(t) + \frac{\theta_i(t) - \theta_A}{R_{ii}} + \sum_{j=1;j\neq i}^{|\mathcal{D}|} \frac{\theta_i(t) - \theta_j(t)}{R_{ij}} = PD_i(t). \quad (2)$$

Here, $C_i$ and $R_{ii}$ denotes the thermal capacitance and self-resistance of device $\delta_i$ respectively, $R_{ij}$ denotes the thermal cross-coupling resistance between device $\delta_i$ and device $\delta_j$, $PD_i(t)$ denotes the instantaneous power dissipated by the device $\delta_i$ at time $t$. $\theta_i(t)$ and $\theta_j(t)$ are the instantaneous temperatures (in K) of the device $\delta_i$ and $\delta_j$ respectively at time $t$. $\theta_A$ is the ambient temperature and is assumed constant throughout time $t$.

Power dissipation by each device at a specific DVFS or frequency configuration $f$ can be decomposed into temperature dependent and independent components [22] as,

$$PD_i^f(t) = \phi_i^f + \psi_i^f \theta_i^f(t). \quad (3)$$

Here $\phi_i^f$ and $\psi_i^f$ are architecture and DVFS specific parameters for device $\delta_i$ running at DVFS configuration $f$ . Replacing $PD_i^f(t)$ in Eq. 2 by its components in Eq. 3 and combining the parameters of all the devices in the platform, we have the following.

$$\begin{aligned}\dot{\theta}_i^f(t) = &\left(\frac{\psi_i^f}{C_i} - \frac{1}{C_i R_{ii}} - \sum_{j=1;j\neq i}^{|\mathcal{D}|} \frac{1}{C_i R_{ij}}\right)\theta_i^f(t) \\ &+ \sum_{j=1;j\neq i}^{|\mathcal{D}|} \frac{1}{C_i R_{ij}}\theta_j^f(t) + \frac{\phi_i^f}{C_i} + \frac{\theta_A}{C_i R_{ii}}.\end{aligned} \quad (4)$$

Converting Eq. 4 with generalised state-space matrices $A_f$ and $B_f$ (time-invariant), we get rate of change of temperature as-

$$\dot{\Theta}^f(t) = A_f \Theta^f(t) + B_f. \quad (5)$$

Here, $\Theta^f(t)$ is the vector of all individual device temperatures $\theta_i^f(t)$, $\forall \delta_i \in \mathcal{D}$ for the frequency configuration $f$. $A_f$ and $B_f$ are $|\mathcal{D}| \times |\mathcal{D}|$ and $|\mathcal{D}| \times 1$ dimensional matrices respectively, composed of architecture specific static parameters for $f$. The terms of matrices $A_f$ and $B_f$ in terms of the thermal coefficients are as follows,

$$a_{ii}^f = \frac{\psi_i^f}{C_i} - \frac{1}{C_i R_{ii}} - \sum_{j=1;j\neq i}^{|\mathcal{D}|} \frac{1}{C_i R_{ij}},\ a_{ij}^f = \frac{1}{C_i R_{ij}},$$



Authorized licensed use limited to: Wuhan University. Downloaded on September 20,2026 at 11:58:48 UTC from IEEE Xplore. Restrictions apply.

$$b_i^f = \frac{\phi_i^f}{C_i} + \frac{\Theta_A}{C_i R_{ii}}$$

If $t_q$ and $t_{q'}$ are the timestamps of two successive thermal measurements ($t_{q'} > t_q$) and $\Delta t_q = t_{q'} - t_q$, solving the vectored differential Eq. 5 using methods like exponential integrators [23], we get the following recurrence relationship.

$$\Theta^f(t_{q'}) = e^{\Delta t_q A_f}\Theta^f(t_q) + A_f^{-1}(e^{\Delta t_q A_f} - I)B_f. \quad (6)$$

In Eq. 6, the parameters $C_i$ and $R_{ij}$, $\forall i, j$ can be assumed constant for a specific compute platform architecture [22] irrespective of kernel types and hence can be pre-determined by training on offline profiled thermal traces. We profile time-series datasets of thermal measurements (or thermal traces) on the target platform $\mathcal{D}$ in each fixed DVFS configuration $f$ separately. For this purpose, we run compute intensive kernels such as matrix multiplication in both the devices simultaneously as well as on each device separately, maintaining $100\%$ utilisation for the executing device(s). When the kernels are running on only one device, the other device is kept idle and the parameters are trained separately for these cases. For every DVFS configuration $f \in \mathcal{F}$, we employ linear regression on Eq. 5 (followed by curve fitting on Eq. 6 for fine-tuning) for deriving the matrices $A_f, B_f$ in Eq. 5. In our thermal model, the parameters are stored corresponding to every configuration in a lookup table and used by the Eq. 6 to predict the end temperature of any kernel if the initial temperature, CPU and GPU execution times and DVFS configuration is known.

The trained $a_{ii}^f$ values are observed to be in the order of $-10^{-6}$ while the values of $a_{ij}^f$ are in the order of $10^{-16}$ for all DVFS configurations. This is because the values of thermal resistance for cross-coupling ($R_{ij}$) are significantly lesser than self-resistance ($R_{ii}$) as the amount of heat dissipated through the chip package (and to the ambient) is much larger than the heat flowing into the adjacent cores [24].

**Observation 1.** *Given the above observation in the platform setting, i.e.,*

$$a_{ii}^f < 0 \text{ and } |a_{ii}^f| \gg \sum_{j=1;j\neq i}^{|\mathcal{D}|} |a_{ij}^f| \quad \forall i, j \in [1 : |\mathcal{D}|], \quad (7)$$

*we infer that the matrix $A_f$ is strictly diagonally dominant with negative diagonal entries.*

With the above assumption considered in our thermal model, we show that the following two lemmas hold true while performing any computation (execution of kernels) with a constant maximum device utilization at a given fixed DVFS configuration ($f$). The constant utilization assumption is true for kernels having compute intensive loops that are executed by most of the threads actually launched. Such divergence free kernels represent a fairly large class. However, in practice, for other kernel types, the execution can be piece-wise divided to satisfy such approximations. Surely, in both cases, we are ignoring very local inter-instruction variations.

**Lemma 1.** *The final temperature of a platform at steady state, would be an asymptotically constant function of the DVFS configuration ($f$), irrespective of the initial temperature of the platform assuming constant maximum device utilization.*

*Proof.* For a strictly diagonally dominant matrix with all the diagonal elements being negative, all the eigenvalues will have negative real parts [25]. Thus, by empirical observations, the eigenvalues of $A_f$ have negative real parts. Then, from standard stability results [26], it follows that $\lim_{\Delta t_q \to \infty} e^{\Delta t_q A_f}$ exists and converges to 0. Substituting $\Delta t_q \to \infty$ in Eq. 6, we get the final temperature at steady state (or asymptotic convergence temperature) with respect to $f$ as,

$$\Theta^f(t_\infty) = \lim_{\Delta t_q \to +\infty} \Theta^f(t_{q'}) = -A_f^{-1}B_f \quad \forall \Theta^f(t_q) \in \mathbb{R}^+ \quad (8)$$

□

The elements of the vectors $\Theta^f(t)$ and $\Theta^f(t_\infty)$ are expressed as $\theta_i^f(t)$ and $\theta_i^f(t_\infty)$; $\forall \delta_i \in \mathcal{D}$ respectively. The maximum element of each vector is expressed using the $\|\cdot\|_\infty$ operator. The current peak platform temperature and the maximum asymptotic convergence temperature are expressed as $\|\Theta^f(t)\|_\infty = \max_{i=[1,|\mathcal{D}|]} \theta_i^f(t)$ and $\|\Theta^f(t_\infty)\|_\infty = \max_{i=[1,|\mathcal{D}|]} \theta_i^f(t_\infty)$.

**Lemma 2.** *The maximum temperature of a device $\delta_i$ in a time duration between $t$ (current time) and $t'$ (any time in the future), will be upper-bounded by the maximum of the current temperature $\theta_i^f(t)$ and the asymptotic convergence temperature $\theta_i^f(t_\infty)$ at a given fixed DVFS configuration $f$.*

*Proof.* To prove this lemma, we consider two cases separately where in the first case, we consider $\theta_i^f(t) > \theta_i^f(t_\infty)$ whereas in the second case we consider that $\theta_i^f(t) \leq \theta_i^f(t_\infty)$.

From Eq. 8, we have $B_f = -A_f\Theta^f(t_\infty)$ which we substitute in Eq. 5 and get, $\dot{\Theta}_f(t) = A_f(\Theta^f(t) - \Theta^f(t_\infty))$. Hence, the temperature gradient of $\delta_i$ at $f$ can be written as,

$$\dot{\theta}_i^f(t) = a_{ii}^f(\theta_i^f(t) - \theta_i^f(t_\infty)) + \sum_{j=1;j\neq i}^{|\mathcal{D}|} a_{ij}^f \cdot (\theta_j^f(t) - \theta_j^f(t_\infty)). \quad (9)$$

$$< a_{ii}^f(\theta_i^f(t) - \theta_i^f(t_\infty)) + \sum_{j=1;j\neq i}^{|\mathcal{D}|} a_{ij}^f \cdot \max_{j=1;j\neq i}^{|\mathcal{D}|} (\theta_j^f(t) - \theta_j^f(t_\infty)).$$

Now, using inequality 7 from Observation 1, we have,

$$\sum_{j=1;j\neq i}^{|\mathcal{D}|} a_{ij}^f \cdot (\theta_j^f(t) - \theta_j^f(t_\infty)) < (-a_{ii}^f) \cdot \max_{j=1,j\neq i}^{|\mathcal{D}|} (\theta_j^f(t) - \theta_j^f(t_\infty)).$$

We assume that all device temperatures are in the operating range at or above ambient temperature ($\theta_A$) but below the tripping point, i.e. $\theta_A \leq \theta_j^f(t) < T_{trip}$. Now for mobile platforms, the acceptable ambient or room temperature lies in the range of $253\,\mathrm{K}$ to $318\,\mathrm{K}$ [27], [28]. For embedded platforms with such a range of ambient temperatures, the following inequality $(T_{trip} - \theta_A) < \theta_A$ holds true. For



Authorized licensed use limited to: Wuhan University. Downloaded on September 20,2026 at 11:58:48 UTC from IEEE Xplore. Restrictions apply.

violating this inequality, even with the lowest ambient temperature 253 K, the trip point needs to be greater than 506 K which is infeasible for any platform. Now, $\theta_A \leq \theta_i^f(t_\infty)$ unless any external cooling agents are used. Thus, $\forall j \in \mathcal{D},\ \theta_j^f(t) - \theta_j^f(t_\infty) < (T_{trip} - \theta_A) < \theta_A < \theta_j^f(t_\infty)$. Now, using the above inequality and our empirical Observation 1, we have, $\sum_{j=1;j\neq i}^{|\mathcal{D}|} a_{ij}^f \cdot (\theta_j^f(t) - \theta_j^f(t_\infty)) \ll (-a_{ii}^f) \cdot \theta_i^f(t_\infty)$. Hence, removing the negligible term in Eq. 9 we get, $\dot{\theta}_i^f(t) \approx a_{ii}^f(\theta_i^f(t) - \theta_i^f(t_\infty))$.

For our first case, when $\theta_i^f(t) > \theta_i^f(t_\infty)$, from Eq. 7 we get $a_{ii}^f < 0$ which implies $\dot{\theta}_i(t) < 0$. This means $\theta_i^f(t)$ decreases monotonically and therefore the maximum temperature of $\delta_i$ in future is upper-bounded by $\theta_i^f(t)$. Similarly, for the second case, when $\theta_i^f(t) < \theta_i^f(t_\infty)$ it implies $\dot{\theta}_i^f(t) > 0$. This means $\theta_i^f(t)$ increases monotonically until it reaches maximum asymptotic convergence temperature and therefore the temperature of $\delta_i$ in future is upper-bounded by $\theta_i^f(t_\infty)$. □

Lemma 1 and 2 help to approximately characterize the thermal model of the platform. In our thermal model, we estimate the peak temperature $\mathcal{T}_k(s)$ of the platform $\mathcal{D}$ for a kernel $\kappa_k$ executing with mapping configuration $s = \{p, f\}$. Based on the value of $p$, the kernel can be partitioned across multiple devices, and it may happen that one of the devices finishes execution before the other. Let us consider $t_0$ and $t_e$ are the start and end time of the kernel execution and $t_c$ is the time up to which both devices are utilised concurrently $[t_0 \leq t_c \leq t_e \leq t_\infty]$. As only one kernel can be scheduled in $\mathcal{D}$ at a given time, from Lemma 2, we may assert that $\mathcal{T}_k(s)$ occurs at time $t_0$ or $t_c$ or $t_e$ and can be expressed as $\max(\|\Theta^f(t_0)\|_\infty, \max(\|\Theta^f(t_c)\|_\infty, \|\Theta^f(t_e)\|_\infty)$. After $t_c$, only one device with unfinished work-items continues, thus changing the constant utilization assumption. Hence it is added as a new start/end/measurement point. Thus, the peak temperature $\mathcal{T}_k(s)$ is given by,

$$\mathcal{T}_k(s) = \max_{i\in[1,|\mathcal{D}|];j\in\{0,c,e\}} \theta_i^f(t_j). \tag{10}$$

### B. MPC Optimizer Design

We summarise the terms used in the following algorithms in Table II. With the proposed thermal and latency models, we first consider the following baseline method for solving Eq. 1.

*1)* ***Exhaustive search using Depth First Branch-and-Bound (DFBB)*** *:* This approach considers the currently executing kernel ($\kappa_c$) and searches for optimal configuration ($s_{opt}$) for the immediate next kernel ($\kappa_{c+1}$) keeping in view the predicted performance of all the kernels up to $\kappa_{c+h}$ in the hyper-period. We represent the complete search-space for $s_{opt}$ using a search tree where the currently executing kernel $\kappa_c$ along with its current configuration creates the root node. Each node corresponding to a kernel $\kappa_i; \forall i \in [c, c+h-1]$ of the tree has $|\mathcal{S}|$ children nodes, one for every possible configuration of the next kernel $\kappa_{i+1}$ in the hyper-period. The node data structure stores predicted information corresponding to a kernel $\kappa_i$ such as start time, end time, final temperature, the maximum platform temperature among all the kernels from the $\kappa_c$ to $\kappa_i$, and additionally a pointer to its parent node to be used for backtracking. All nodes at the same kernel level $KL_i$ of the search tree correspond to a single kernel $\kappa_i$. The search tree unfolds up to the leaf nodes at kernel level $KL_{c+h}$, from where it backtracks up to the root. We identify the leaf node with the minimum value of the minimization objective in Eq. 1. As we backtrack from that leaf node up to the root node, the node at each kernel level $KL_j$ gives the mapping configuration of the corresponding kernel $\kappa_j$. We define the following terminologies that are used next.

| Symbol | Description |
|---|---|
| $KL_i$ | Level of all nodes corresponding to kernel $\kappa_i$ in the search tree. |
| $LST(\kappa_c, f)$ | Latest start time for kernel $\kappa_c$ to satisfy all kernel deadlines assuming execution of current and future kernels at a constant DVFS configuration $f$ (Property of kernel level). |
| $CMT(\kappa_c, s)$ | Current maximum platform temperature from root node up to a node at level $KL_c$ (property of a node). |
| $CBT(\kappa_c)$ | Current best temperature for kernel $\kappa_c$ is the minimum of CMT values over all nodes at $KL_c$ (Property of kernel level). |
| $BSST$ | Best steady-state peak temperature achieved at any time during the search assuming all unexplored kernels run at any fixed DVFS configuration (Dynamic property of search tree). |
| $LH$ | Latency Headroom is the difference between the finish time of current kernel and minimum LST of the next kernel (Node property). |
| $Fitness$ | Fitness score of a node is the weighted sum of LH and (BSST - CBT) representing the likelihood of the node being optimal. |
| $NodeVectors$ | Data structure containing the search tree. Vector of arrays indexed by the kernel level. |
| $Speed_{c,f}$ | Ratio between the total number of work-items in $\kappa_c$ and the actual time taken to execute $\kappa_c$. |
| $Speedup_{c,f}$ | Ratio of the speed of a kernel $\kappa_c$ at frequency $f$ and the speed while executing at the minimum frequency. |

Table II: List of used symbols

**Definition 1.** *Current Best Temperature or $CBT$ for a kernel $\kappa_c$ is defined as the minimum value of the maximum platform temperature which can be attained by executing all the kernels from the start of the hyper-period up to the currently searched kernel $\kappa_c$ at any mapping configuration $s \in \mathcal{S}$.*

**Definition 2.** *Current Maximum Temperature or $CMT$ for a kernel $\kappa_c$ at a specific configuration $s \in \mathcal{S}$ is defined as the maximum platform temperature at the end of execution of $\kappa_c$.*

**Definition 3.** *Latest start time or $LST(\kappa_c, f)$ of a kernel $\kappa_c$ in a hyper-period $H$ for a DVFS configuration $f$ is defined as the latest start time of the kernel $\kappa_c$ in the hyper-period, for which $\exists p_k \in \mathcal{P}$, such that all kernels ($\kappa_k$) from $\kappa_c$ to the last kernel in $H$, when executed with mapping configuration $s = (p_k, f)$, will not violate any future kernel deadline.*

$LST$ of each kernel is computed and stored in a table at the beginning of the hyper-period when the task set changes dynamically and is used in both the approaches in Sec. IV-B1 and IV-B2. LST accounts for all of the future kernel deadlines. The bottom-up recursive update rule for the LST is $\forall i \in [1, |\mathcal{K}^H|]$, $LST(\kappa_i, f) \leftarrow \min(LST(\kappa_{i+1}, f), d_i) - \min_{p\in\mathcal{P}} l_i(p, f)$.

The $LST$ table is passed as input to the driver function (Algorithm 1) of the MPC optimizer that runs for each kernel $\kappa_c$ in the hyper-period to find the optimal mapping configuration ($s_{opt}$) for its next kernel ($\kappa_{c+1}$). It calculates the end time, end temperature using $FindEndTemp()$, and $CMT$ from the established latency and thermal models and initialises the



Authorized licensed use limited to: Wuhan University. Downloaded on September 20,2026 at 11:58:48 UTC from IEEE Xplore. Restrictions apply.

global variable $CBT$. For every mapping configuration $s \in \mathcal{S}$, $\sum_{k=c+1}^{c+h} \mathcal{T}_k(s)$ is computed using $DFBB$ function depicted in Algorithm 2. It finally returns the mapping configuration $s$ for which $\sum_{k=c+1}^{c+h} \mathcal{T}_k(s)$ is found to be the minimum.

**Algorithm 1:** Driver Function

**Input:** Presently executing kernel index $c$, Present mapping config $s_c$, Time elapsed since hyper-period start $t_c$, Present platform temperature $\Theta^f(t_c)$, $LST$
**Data:** Thermal Model, Latency Model
**Output:** Configuration $s_{c+1}$ for the immediate next kernel $\kappa_{c+1}$

$t_e \leftarrow t_c + l_c(s_c)$;
$\Theta^f(t_e) \leftarrow FindEndTemp(s_c, \Theta^f(t_c), l_c(s_c))$;
$CMT \leftarrow \|\Theta^f(t_e)\|_\infty$; $\quad CBT \leftarrow \infty$,
return $\arg\min_{s \in \mathcal{S}} DFBB(s, c+1, c, t_e, \Theta^f(t_e), CMT, LST)$

The recursive DFBB algorithm is depicted in Algorithm 2. Each node at kernel level $KL_c$ checks if the predicted end time of the present kernel is less than the $\min_{f \in \mathcal{F}} LST(\kappa_{c+1}, f)$ and if the maximum temperature observed is less than the $CBT$. If both of these conditions are satisfied, the node recursively spawns children nodes for all configurations of the next kernel. This goes on until any leaf node is reached, which indicates a solution is found. It then updates the $CBT$ as the maximum platform temperature encountered till the last kernel within the deadline. Once all the solutions are found, it returns the minimum of the overall peak temperatures observed for each of the children nodes for post-recursive backtracking.

**Algorithm 2:** Recursive DFBB Function

**Input:** Configuration $s$, Presently searched kernel index $c$, Presently executing kernel index $c_{init}$, Time elapsed since hyper-period start ($t_c$), Current platform temperature ($\Theta^f(t_c)$), CMT, LST
**Data:** Thermal Model, Latency Model, Horizon length $h$, $CBT$ (Global Variable)
**Output:** Minimum Peak Temperature ($\mathcal{T}_k$) for all future kernels up to horizon length $h$

$t_e \leftarrow t_c + l_c(s)$;
**if** $t_e \geq \min_{s \in \mathcal{S}} LST(\kappa_c, f)$ **then**
  return $\infty$;
**else**
  $\Theta^f(t_e) \leftarrow FindEndTemp(s, \Theta^f(t_c), l_c(s))$;
  $CMT \leftarrow \max(CMT, \|\Theta^f(t_e)\|_\infty)$;
  **if** $CMT \geq CBT$ **then**
    return $\infty$;
  **else if** $c == c_{init} + h$ **then**
    $CBT \leftarrow \min(CBT, CMT)$;
    return CMT ;
  **else**
    $T_{MIN} \leftarrow \min_{s' \in \mathcal{S}} DFBB(s', c+1, t_e, \Theta^f(t_e), CBT, LST)$
    return $\max(T_{MIN}, CMT)$ ;

**Time Complexity:** In the recursive Algorithm 2, each node generates $|\mathcal{S}|$ children nodes, therefore the running time can be expressed as $T(n) = |\mathcal{S}| \cdot T(n-1) + \mathcal{O}(1)$, $\forall n \in [1, h]$ and $T(0) = \mathcal{O}(1)$. Hence, Algorithm 1 has a worst-case running time complexity of $\mathcal{O}((F_C \cdot F_G \cdot P)^h)$. The exponential time complexity of complete state-space search is a major drawback that limits the value of $h$ to a very low value (In our target platform, it is 3 if the workload is in the magnitude of msec.).

*2) **Heuristic Iterative Exploration using Beam Search**:* As an improvement over the previous method, we propose the following heuristic based, light-weight, beam search algorithm for solving the MPC in polynomial time. Algorithm 3 uses a modified beam search approach to roughly estimate the goodness of a predicted solution without exhaustive search.

We use our notion of upper bound for the peak platform temperature as a pruning mechanism here. If at a given time $t_0$, all upcoming kernels are executed with a fixed DVFS configuration $f$ and the temperature of the platform at $t_0$ is less than the maximum steady-state temperature $\|\Theta^f(t_\infty)\|_\infty$, the peak temperature of the platform throughout the remainder of the execution will be upper-bounded by $\|\Theta^f(t_\infty)\|_\infty$ (from Lemma 2). This holds true irrespective of the partition ratio, as well as the number or types of kernels left to be executed. If the start time of $\kappa_c$ is less than $LST(\kappa_c, f)$, it can be stated with certainty that at least one configuration mapping exists for which the minimization objective will never exceed $\|\Theta^f(t_\infty)\|_\infty$, without exploring all the way up to the leaf node.

This property is utilized to determine the most promising solutions at each kernel level and accordingly limit the exploration of a search tree. The nodes are ranked at each kernel level on the basis of a fitness score computed from predicted latency and thermal performance (defined later). Higher fitness for a node $v$ in the search tree indicates a greater chance of finding a solution closer to optimal in the subsequent children nodes of $v$. The beam search retains $W$ (beam width) most promising nodes for further exploration at each kernel level and prunes the rest. Thus it maintains a fixed number of nodes at each level and ensures polynomial time completion at the cost of slightly sub-optimal results compared to exhaustive search. $W$ is fixed for Algorithm 3 and chosen suitably to balance the trade off between the overhead and accuracy in the optimization problem. We define the following terminologies that have been used in Algorithm 3.

**Definition 4.** *Given a search tree, the Best Steady-State Temperature (BSST) at any instant during the search is given by the predicted minimum value of the objective (given in Eq. 1) which is computed assuming all the unexplored future kernels execute in a constant DVFS configuration.*

$BSST$ is updated by any node $v$ corresponding to $\kappa_c$ in the search tree to the min of the previous value of BSST, and the least steady-state temperature $\|\Theta^f(t_\infty)\|_\infty$ ; $\forall f \in \mathcal{F}'$ where $\mathcal{F}' \subset \mathcal{F}$ such that $\forall f' \in \mathcal{F}'$, $LST(\kappa_{c+1}, f') \geq v.EndTime$.

**Definition 5.** *Latency Headroom or $LH$ of a node $v$ corresponding to a kernel $\kappa_c$ in the search tree can be defined as the difference between the predicted end time of the node and the minimum value of $LST(\kappa_{c+1}, f)$, $\forall f \in \mathcal{F}''$ where $\mathcal{F}'' \subset \mathcal{F}$ such that $\forall f'' \in \mathcal{F}''$, $BSST \geq \|\Theta^{f''}(t_\infty)\|_\infty$.*

**Definition 6.** *Fitness score of a node $v$ in the search tree is a weighted sum of the latency headroom ($LH$) and thermal headroom ($BSST$ - v.EndTemp).*

The maximum temperature of the platform is lower-bounded by $CBT$ and upper bounded by $BSST$. As the search proceeds and more kernels are explored, $BSST$ reduces while $CBT$ increases. When $CBT \geq BSST$, the solution cannot be improved any further and hence is terminated.



Authorized licensed use limited to: Wuhan University. Downloaded on September 20,2026 at 11:58:48 UTC from IEEE Xplore. Restrictions apply.

**Algorithm 3:** Look-ahead Beam-Search Optimizer

**Input:** Index of currently executing kernel $c$, Current configuration $s_c$, Time elapsed since hyper-period start ($t_c$), Current platform temperature $\Theta^f(t_c)$, Horizon length ($h$), Beam width $W$, $LST$
**Data:** Thermal Model, Latency Model
**Output:** Configuration $s_{c+1}$ for the immediate next kernel $\kappa_{c+1}$

1 Root $\leftarrow CreateNode(\kappa_c, t_c, s_c, \Theta^f(t_c), NULL)$
2 Declare $NodeVectors[h]$ (Array of $h$ vectors of Node)
3 Push Root into $NodeVectors[0]$
4 BSST $\leftarrow \infty$; $BestConfig \leftarrow -1$;
5 **for** *Configuration* $f \in \mathcal{F}$ **do**
6  **if** $LST(\kappa_{c+1}, f) \geq t_c + L_c(s_c)$ **then**
7  **if** $\|\Theta^f(t_\infty)\|_\infty < BSST$ **then**
8  $BSST \leftarrow \|\Theta^f(t_\infty)\|_\infty$ $BestConfig \leftarrow f$
9 **if** $BestConfig == -1$ **then**
10  return *Failure* ; // No solution is feasible
11 **for** $i \in range[c+1, c+h]$ **do**
12  $CBT \leftarrow \infty$
13  **for** *each previous node* $v' \in NodeVectors[i-1-c]$ **do**
14  **for** *Configuration* $s \in \mathcal{S}$ **do**
15  $v = CreateNode(\kappa_i, v'.EndTime, s_i, v'.EndTemp, \&v')$
16  **if** $v.EndTime \leq \max_{f \in \mathcal{F}} LST(\kappa_{i+1}, f)$ **then**
17  $CBT \leftarrow \min(CBT, v.MaxTemp)$
18  **if** $v.EndTemp \leq BSST$ **then**
19  **for** *configuration* $f \in \mathcal{F}$ **do**
20  **if** $v.EndTime \leq LST(\kappa_{i+1}, f)$ **then**
21  $BSST \leftarrow \min(BSST, \max(v.EndTemp, \|\Theta^f(t_\infty)\|_\infty)$
22  $LH \leftarrow ComputeLH(BSST, LST, v.EndTime)$
23  $v.Fitness \leftarrow FindFitness(LH, BSST, v.EndTemp)$
24  Push $v$ in $NodeVectors[i-c]$
25  **if** $NodeVectors[i-c].Size == 0$ **then**
26  return Success (Return due to no scope of further improvement)
27  Sort $NodeVectors[i-c]$ in descending order of Fitness and resize to W
28  Root $\leftarrow$ Recursive backtrack up to root node of $NodeVectors[i-c][0]$
29  Update $BestConfig \leftarrow Root.Config$
30  **if** $CBT \geq BSST$ **then**
31  return Success (Return due to no scope of further improvement)
32 return Success (Return due to exceeding horizon);

Similar to our previous exhaustive search approach, LST table is populated at the beginning of the hyper-period when MPC is triggered. Algorithm 3 runs for each kernel in the hyper-period. It starts with the function $CreateNode$ which encapsulates the thermal and latency models. The function takes the following inputs- kernel $\kappa_c$, the actual/predicted start time of $\kappa_c$ ($t_c$), a mapping configuration $s$, actual/predicted initial temperature of $\kappa_c$ ($\Theta^f(t_c)$) and a pointer to the previous node. It internally computes the predicted end temperature ($EndTemp$), end time ($EndTime$) and returns a node of the search-tree containing all this information (Line# 1). The node thus obtained is inserted into a dynamic array of a vector of nodes indexed by the kernel level in the search tree (Line# 2-3). Next, we initialise the global variables $BestConfig$ and $BSST$ for the current kernel (Line# 4-9).

In case of an infeasible solution, the algorithm terminates as any mapping configuration will lead to at least one deadline violation (Line# 10-11). Else, at every kernel level, for all the nodes in the vector, children nodes are generated for different mapping configurations for the currently searched kernel $i$ using $CreateNode$ function (Line# 12-16). The $CBT$ is defined separately for each kernel level. $CBT$ is updated to the least maximum temperature of the newly generated nodes if their end times do not exceed $\min_{f \in \mathcal{F}} LST(\kappa_{i+1}, f)$ (Line# 17-18). All the other nodes that do not satisfy the above deadline adherence criteria or have an end temperature greater than the $BSST$, are pruned and not searched.

From each node, the search explores all DVFS configurations, and the $BSST$ is updated (Line# 19-23). Thereafter, the latency headroom is computed w.r.t mapping configurations using $ComputeLH$ function that computes LH (Line# 24). The fitness score is computed for all the new nodes using $FindFitness$ function which takes the LH and the difference between BSST and predicted node end temperature as input (Line# 25). The nodes are inserted into the NodeVector for the current level.(Line# 26)

If there are no new nodes, the search is terminated (Line# 27-28). Else, the new nodes are sorted in decreasing order of fitness score and the first W (Beam-Width) nodes are inserted into the NodeVector for the current level while the others are discarded (Line# 29). The configuration of the node at kernel level $\kappa_{c+1}$, which is an ancestor to the node at the current kernel level with the highest fitness score, is updated as the current best configuration (Line# 30-31). Therefore, at each level, $W \cdot F_C \cdot F_G \cdot P$ nodes are generated, out of which $W$ nodes are kept and the rest are discarded, allowing faster execution. At any level, if the $BSST$ is less than or equal to $CBT$, the search terminates successfully (Line# 32-33).

**Time Complexity:** In Algorithm 3, the outer loop (Line# 12-33) executes $\mathcal{O}(h)$ times, once for each of the kernel levels searched. The intermediate loop (Line# 14-26) executes $\mathcal{O}(W)$ times as $NodeVector$ is always resized to $W$ size for the previous kernel whereas, the inner loop (Line# 15-26) executes $\mathcal{O}(|\mathcal{S}|)$ times and the innermost loop (Line# 20-23) executes $\mathcal{O}(|\mathcal{F}|)$ times in the worst case. Therefore, the overall time complexity of the algorithm would be $\mathcal{O}(|\mathcal{F}| \cdot |\mathcal{S}| \cdot W \cdot h)$.

### C. Low level Controller

The LLCs for frequency tuning use pole-placement based self-tuning control techniques [29], [30] which dynamically model the *speedup* of a task as a function of task latency at different clock frequency values. LLC for device $\delta_x$ takes as input, the actual execution time of a kernel $\kappa_c \in \mathcal{K}^H$, computes the remaining headspace for the next kernel $\kappa_{c+1}$, and outputs the discrete frequency setting $f_x$ for $\delta_x$ for $\kappa_{c+1}$. This is repeated iteratively for all kernels in $H$.

We define the speed of a kernel $\kappa_c$ as the ratio between the total number of work-items in $\kappa_c$ and the actual time taken to execute $\kappa_c$, whereas the target speed is calculated by considering the number of work-items in $\kappa_c$ and the time left till the deadline of $\kappa_c$. The difference in the target and the actual speed indicates the term $error$ which is adjusted by the controller. Since speedup of a task is an abstraction of the relative changes in the performance with respect to frequency variation, we define $speedup$ as the ratio between the speed of the task at frequency $f$ and the speed of the task when it is executing at the minimum frequency setting ($speed_{base}$). Our objective is to calculate $speedup_{c+1}$ which is the required speedup for executing the next kernel in the hyper-period based on



Authorized licensed use limited to: Wuhan University. Downloaded on September 20,2026 at 11:58:48 UTC from IEEE Xplore. Restrictions apply.

the measured speedup ($speedup_c$) of the current kernel. This is obtained by using the control theoretic speedup equation $speedup_{c+1} \leftarrow speedup_c + ((1-\rho)/speed_{base}) \times error$ where $\rho$ is the pole of the controller. From an offline created hash table for storing speedup for all frequency settings for all devices, the required frequency $f'$ for attaining $speedup_{c+1}$ is applied to $\kappa_{c+1}$.

## V. Evaluation

### A. Target Platform and Runtime Framework

We have performed our evaluations on an Odroid-XU4 embedded platform [31] having the Samsung Exynos 5422 SoC which comprises of the following compute elements- (i) a quad-core ARM Cortex-A7 (LITTLE CPU), (ii) a quad-core ARM Cortex-A15 (big CPU), and (iii) an ARM Mali-T628 GPU with 6 Streaming Multiprocessors (SMs) as depicted in Fig. 4. The platform supports DVFS for each compute element and is equipped with thermal sensors for big CPU and GPU.

Our proposed framework leverages the asynchronous event-driven programming model supported by OpenCL. For an OpenCL application executing on the CPU, the role of the CPU cores switch between host and compute devices. So, we limit the OpenCL compute device for CPU to the big CPU cores only and use the LITTLE CPU cores for all other services such as scheduling and temperature monitoring. In the paper, we use the terms CPU device, A15 cluster, or big CPU interchangeably.

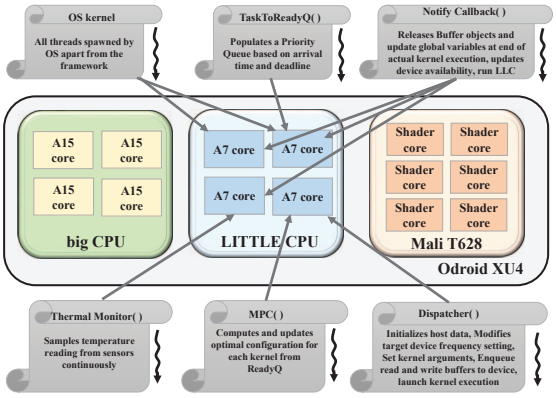



Figure 4: Runtime framework on Odroid-XU4 platform

The Odroid-XU4 architecture along with the thread mapping used in our framework is depicted in Fig. 4. We bind the OS (Ubuntu 18.04 LTS ) to the first two cores of the LITTLE CPU by modifying the OS boot file settings. Using a suitable specification file, our framework can automatically generate OpenCL host code for launching kernels following scheduling decisions. When a kernel is dispatched for scheduling, the framework internally sets the kernel arguments, target device, task partition, orchestrates host-device data-transfer, and memory release, and also triggers low level DVFS controllers. We map the threads corresponding to the *dispatcher* (a complex host program for dispatching tasks at a given configuration) and MPC optimization code to the fourth core of the LITTLE CPU by setting the thread affinity during execution. The *Thermal Monitor* thread discretely samples the temperature readings from the sensors with a sampling period of 30 ms. While temperature spikes inside 30ms interval may be missed, their impact on the continuous nature of thermal dynamics gets captured in the subsequent samples. From the extensive training and profiling phase, such eventual impacts are expected to be learned by the models. Another thread *TaskToReadyQ*, is mapped to the third and second core of the LITTLE CPU respectively. TaskToReadyQ pushes the kernels ready to be dispatched onto *Ready Queue* as soon as the task's arrival time has been reached and its predecessor task in the DAG has finished execution. Ready Queue is implemented as a priority queue that returns the task with the earliest deadline when dequeued. The asynchronous callback function *Notify Callback*, which is instantiated when a device becomes free after each kernel execution, is used to release memory objects related to the kernel, update global variables for performance measures, and to run the LLCs. The global variables accessed in such a multi-threaded framework are maintained using suitable mutex locks. The MPC controller runs on the fourth core when invoked. When a kernel $\kappa_c \in \mathcal{K}^H$ is dispatched and executed in the target device ($\delta_C$ or $\delta_G$), the MPC optimizer runs in parallel on the LITTLE CPU.

### B. Results

**Task Set:** Our task set contains candidate kernels from typical neural network (NN) based detection pipelines from the automotive domain. In the current framework, we shall assume that all the tasks (i.e. kernels) that can arrive within any hyper-period are known and profiled in advance. It is only their periodicity and thus relative order of arrival as well as the relative deadline which is subject to change at runtime, thus demanding dynamic resource allocation.

We have considered multiple detection pipelines with EdLeNet CNN architecture [32] having soft deadlines for evaluation. Keeping the CNN architecture same, multiple object detection pipelines are created by offline training of the weight matrices for detecting different objects. For detecting multiple objects on the same image, each layer is iterated multiple times with different weight matrices. We have evaluated the execution of 20 concurrent end-to-end object detection tasks in two possible settings of periodicity and deadline, to vary the throughput requirement per unit time.). In the first setting $R$, the periodicity is set such that the deadlines are relaxed (10 object detections have periodicity of $20s$ and the remaining have periodicity of $40s$.). This means we have two CNN pipelines with 9 layers where each layer (one kernel per layer) iterates on 10 weight matrices at periodicities $20s$ and $40s$ respectively. Similarly, in the second setting $S$, the deadlines are kept stringent by reducing their periodicity (2 sets of 10 identical CNN pipelines have periodicity of $20s$.). We have considered the following schemes for comparison purposes.

1) ***Default DTM in Linux***: CPUfreq or CPU speed scaling allows the clock speed of the processor to be adjusted on the fly thus enabling the processor to run at a reduced clock speed to save power. The rules for shifting frequencies, whether to a faster or slower clock speed, and



Authorized licensed use limited to: Wuhan University. Downloaded on September 20,2026 at 11:58:48 UTC from IEEE Xplore. Restrictions apply.

when to shift frequencies, are defined by the *CPUfreq governor*. Default DTM is controlled by *On-demand* CPUfreq governor which follows a race-to-idle scheme. It sets the highest possible frequency while executing any kernel until a specified trip point temperature (90° C) has been reached. Thereafter, it abruptly throttles back to a lower configuration until the temperature reduces.

2) ***Baseline MPC with LLCs***: The MPC approach for scheduling using exhaustive state-space search and back-tracking using algorithms described in Sec. IV-B1 and then fine tuned using LLC for each device .
3) ***Heuristic based MPC with LLCs***: The decision made by heuristic based MPC described in Sec. IV-B2 are fine tuned using LLC for each device.
4) ***Static optimization over the whole hyper-period***: The overall optimization is solved statically as one instance for the entire $\mathcal{K}^{H'}$ and mapping configurations are followed.
5) ***Adaptive dynamic thermal management (A-DTM) [10]***: A-DTM mitigates dynamic violations of some given thermal threshold by adaptively tuning task mapping configurations based on performance measures from the previous hyper-period. This framework also targets OpenCL workloads and uses a proportional-integral (PI) DFVS controller along with other actions like task migration and idle slot insertion for minimizing thermal violations adaptively. However, the approach is future unaware and takes multiple hyper-periods to reduce peak temperature.

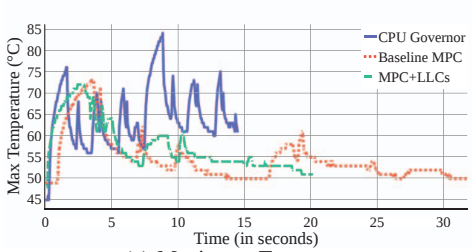
We summarise the thermal profile (maximum and average platform peak temperature) and lateness % over a hyper-period $H$ using the different scheduling approaches with different parameters in Table III. Here the Max. lateness % column captures $\max_{k=1}^{|\mathcal{K}|^H}(e_k - d_k)/d_k \times 100$ (where $e_k$, $d_k$ are the end time and deadline of kernel $\kappa_k$). Max. Tmp. and Avg. Tmp. columns are for the maximum and average of the peak platform temperature over the hyper-period $H$. The sub-columns $R$ and $S$ considers the $R$ state (Relaxed deadlines) and $S$ state (Stringent deadlines) respectively. We also vary the horizon length $h$ and beam width $W$ in our proposed heuristic based MPC for comparison.

| Approaches | Max. Lateness (%) | | Max. Tmp. (°C) | | Avg. Tmp. (°C) | |
|---|---|---|---|---|---|---|
| | R | S | R | S | R | S |
| **Default DTM by OS** | -65 | -65 | 88 | 88 | 79.4 | 80.0 |
| **Static optimization** | -12 | 5 | 68 | 73 | 62.2 | 64.5 |
| **A-DTM** | -24 | -10 | 72 | 79 | 66.4 | 69.4 |
| **Baseline MPC** ($h = 3$) | -8 | 20 | 60 | 71 | 55.1 | 63.8 |
| **Heuristic MPC** ($h = 3$, $W = 3$) | -32 | -0.5 | 64 | 72 | 57.8 | 61.8 |
| **Heuristic MPC** ($h = 5$, $W = 5$) | -27 | 0.7 | 61 | 72 | 56.2 | 64.8 |
| **Heuristic MPC** ($h = 5$, $W = 3$) | -32 | 0.1 | 62 | 71 | 58.1 | 63.8 |
| **Heuristic MPC** ($h = 7$, $W = 3$) | -30 | 0.1 | 62 | 72 | 58.0 | 65.2 |

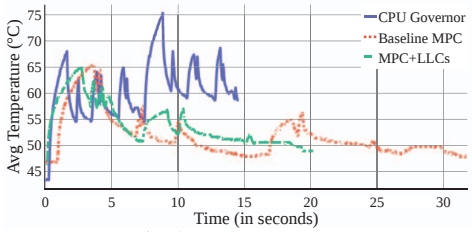
Table III: Comparative evaluation

We observe that the default DTM scheme by OS finishes execution of all kernels at maximum frequency leading to negative lateness for both $R$ and $S$ settings but results in very high peak temperature (85°C). The other approaches are aware of deadlines and perform better in terms of reducing lateness. For the static optimization problem, the measured maximum peak temperature over the hyper-period is much more than the predicted peak temperature (61°C ). This is because the static solution can not factor in the error between the predicted and measured performance while actually executing each kernel on the platform. Similarly, A-DTM also results in higher maximum and average temperatures w.r.t. our approach. The reason behind such observation is that the future unaware control actions by A-DTM are not applied immediately in the hyper-period but are reflected in the next hyper-period.

For our baseline MPC approach, the maximum peak temperature is the lowest among all approaches. This is because the optimization is solved using exhaustive search which also results in the worst lateness performance. Even though the time for executing each kernel was less for baseline, the overhead of recursive MPC calculation with exhaustive search before the start of each kernel leads to a very high overhead, which in turn results in deadline violation and increased lateness. Using our proposed heuristic based MPC, we observe that the maximum and average peak temperature over the hyper-period is slightly more than that of the baseline MPC. However, the negative lateness indicates that no deadline is violated in $R$ setting. However, for $h = 5$ and $W = 5$, the max lateness ($> 0$) is the highest among all the heuristic MPC approaches. When we increase the $W$ keeping $h$ fixed, we observe that lateness increases, and the maximum and average peak temperatures also improve slightly. We observe that keeping $W$ fixed, increasing $h$ beyond 5, the heuristic MPC does not improve thermal or lateness performance. This is because the number of searches for optimal solution per kernel is limited by $W$.



(a) Maximum Temperature



(b) Average Temperature

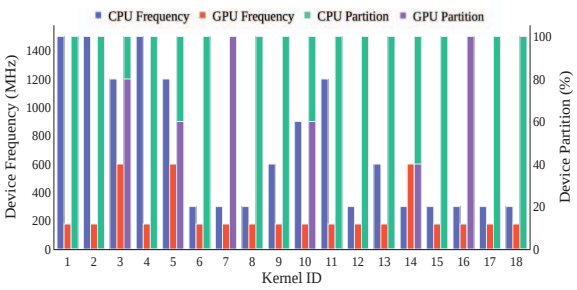
Figure 5: Highest and Average observed temperatures

The maximum and average platform thermal trajectories throughout the hyper-period for setting $S$ are reflected in Fig. 5 where the x-axis plots the time and y-axis plots the platform temperature. For baseline MPC, we set $h = 3$ and for heuristic we set $W = 5, h = 5$. For this configuration, the total overhead of the baseline MPC is 75.6% and that of the heuristic MPC +

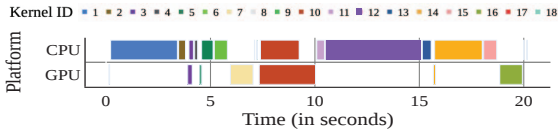


Authorized licensed use limited to: Wuhan University. Downloaded on September 20,2026 at 11:58:48 UTC from IEEE Xplore. Restrictions apply.

LLC is 9.5% of the total time taken for executing all kernels in the hyper-period. With higher $h$, the base MPC will induce even more overhead leading to more lateness. As depicted in the figure, we observe MPC approaches result with a much smoother thermal profile. This indicates more balanced task mapping, leading to uniform heating of the entire platform.




(a) Kernel wise configurations




(b) Gantt chart

Figure 6: Actual Execution and decision taken by MPC+LLCs

The partition ratio and device frequency setting for each kernel in a hyper-period while following the proposed heuristic MPC approach with LLCs over the hyper-period with $W = 5$, $h = 5$ are depicted in Fig. 6a. In the figure, x-axis plots the kernels ranging from $\kappa_1$ to $\kappa_{18}$ indexed by kernel id $1 - 18$, whereas the left-hand y-axis plots the frequency and right-side y-axis plots kernel partitioning ratio. The blue and red vertical bars represent the CPU and GPU frequencies respectively whereas, the green and violet vertical bars represent the partitioning ratio of each kernel mapped to CPU and GPU respectively. In Fig. 6b, we show the Gantt Chart, where execution time taken by different kernels on CPU and GPU are represented using rectangle blocks and the different colours indicate the different kernels in the hyper-period (indexed by kernel id $1 - 18$).

## VI. Related Work

In the recent past, several CPU-GPU thermal management techniques [33], [34], [35], [36] have emerged, but mostly for non-real time systems. Singla et al. [34] proposed a thermal model based proactive DTM technique to prevent thermal violations. Few works like [37], [13] have explored thermal aware real-time task scheduling in CPU-GPU integrated systems, but they mostly explore static scheduling and do not take care of dynamic scenarios. Dynamic scheduling for resource mapping while trading off between different performance measures such as power dissipation, temperature, and latency has been explored using different optimization techniques based on heuristics [38], [39], machine learning (ML) [34], [13], [40], [12], and control theoretic techniques [41], [42], [42], [14]. Control theoretic approaches using feedback to learn system dynamics automatically at run time are able to adjust resource usage based on the difference between measured and expected behaviour of resource usage statistics [43], [29], [14]. However, none of these approaches account for expected future behaviour when making current scheduling decisions.

Many of the future aware approaches in existing literature for resource scheduling use statistical modelling techniques to predict future state(s) of the system corresponding to the present input configuration. Yeo et al. [44] developed a holistic thermal model incorporating both application and core specific features and utilized it to predict prospects of thread migration. The work in [45] proposed a predictive DTM algorithm that exploits performance optimisation opportunities while ensuring operation below the temperature limit.

Future aware scheduling using MPC has also been proposed in the recent past. An explicit MPC based power management technique for homogeneous multi-core platforms was proposed by Zanini et al. [17] which considers pre-computed MPC solutions stored in a lookup table for taking smooth thermal control actions that minimize the variance in performance. Bartolini et al. [46] proposed a distributed, self-calibrating MPC based thermal management technique that exploits performance optimisation opportunities while ensuring operations below the temperature limit for homogeneous multi-core systems. None of these works, however, focus on minimizing the platform temperature, while at the same time, ensuring the performance constraint, especially in CPU-GPU heterogeneous platforms. Moreover, the computational overhead of such MPC based approaches is significant and not suitable for real-time tasks. In [47], the authors use MPC to maximize the energy efficiency in CPU-GPU heterogeneous platforms without compromising performance. Their approach determines a search order to optimize the power consumption of future kernels reducing the complexity from exponential to polynomial time. An approach for power/energy minimization is not directly applicable in the thermal setting since total power consumption will remain equal in different task orders with the same frequency settings. The temperature has a more direct dependence on initial values for each task, which violates such an order invariance property.

## VII. Conclusion

We believe this work is the first *future aware* thermally optimizing scheduling framework for heterogeneous CPU/GPU embedded architectures. As test scenario, we considered multiple data parallel CNN pipelines, exploiting run-time partitioning based heterogeneous programming model with real time deadline constraints. While existing works mostly consider DVFS as the only thermal-aware control action, we extend the OpenCL runtime framework with the flexibility of performing DVFS as well as task partitioning and migration as platform-level control actions to minimize the platform peak temperature over a hyper-period while supporting dynamic periodicity change. Future work includes improving the MPC with lookup tables for predicting latency with resource contention aware analytical latency models [48]. Integrating task mapping options like coarsening [49] and kernel fusion [50] can be another promising avenue to improve resource utilisation.



Authorized licensed use limited to: Wuhan University. Downloaded on September 20,2026 at 11:58:48 UTC from IEEE Xplore. Restrictions apply.

## REFERENCES


[1] Tesla. (May 2022) Chronology of Defect Determination. [Online]. Available: https://static.nhtsa.gov/odi/rcl/2022/RMISC-22V296-1470.pdf

[2] H. Zhang and H. Hoffmann, "Maximizing Performance Under a Power Cap: A Comparison of Hardware, Software, and Hybrid Techniques," in *ASPLOS*. ACM, 2016, p. 545–559.

[3] K. Skadron, "Hybrid architectural dynamic thermal management," in *DATE*. IEEE, 2004, pp. 10–15.

[4] H. Hanson, S. W. Keckler, S. Ghiasi, K. Rajamani, F. Rawson, and J. Rubio, "Thermal response to DVFS: Analysis with an Intel Pentium M," in *ISLPED*. IEEE, 2007, pp. 219–224.

[5] K. Lampka and B. Forsberg, "Keep it slow and in time: Online DVFS with hard real-time workloads," in *DATE*. IEEE, 2016, pp. 385–390.

[6] T. Chantem, X. S. Hu, and R. P. Dick, "Temperature-Aware Scheduling and Assignment for Hard Real-Time Applications on MPSoCs," *IEEE Transactions on Very Large Scale Integration Systems*, vol. 19, no. 10, pp. 1884–1897, 2011.

[7] Y. Fu, N. Kottenstette, C. Lu, and X. D. Koutsoukos, "Feedback thermal control of real-time systems on multicore processors," in *Proceedings of the tenth ACM international conference on Embedded software*, 2012, pp. 113–122.

[8] Y. Ge, P. Malani, and Q. Qiu, "Distributed Task Migration for Thermal Management in Many-Core Systems," in *DAC*. IEEE, 2010, p. 579–584.

[9] Y. G. Kim, M. Kim, J. M. Kim, and S. W. Chung, "M-DTM: Migration-based dynamic thermal management for heterogeneous mobile multi-core processors," in *DATE*. IEEE, 2015, pp. 1533–1538.

[10] S. Maity, A. Ghose, S. Dey, and S. Biswas, "Thermal-Aware Adaptive Platform Management for Heterogeneous Embedded Systems," *ACM Trans. Embed. Comput. Syst.*, vol. 20, no. 5s, sep 2021.

[11] D. Goswami, A. Masrur, R. Schneider, C. J. Xue, and S. Chakraborty, "Multirate controller design for resource-and schedule-constrained automotive ecus," in *DATE*. IEEE, 2013, pp. 1123–1126.

[12] A. Ghose, S. Maity, A. Kar, and S. Dey, "Orchestration of Perception Systems for Reliable Performance in Heterogeneous Platforms," in *DATE*. IEEE, 2021.

[13] S. Isuwa, S. Dey, A. K. Singh, and K. McDonald-Maier, "TEEM: Online Thermal-and Energy-Efficiency Management on CPU-GPU MPSoCs," in *DATE*. IEEE, 2019, pp. 438–443.

[14] S. Maity, A. Ghose, S. Dey, and S. Biswas, "Thermal load-aware adaptive scheduling for heterogeneous platforms," in *VLSID*. IEEE, 2020, pp. 125–130.

[15] H. Amrouch, S. B. Ehsani, A. Gerstlauer, and J. Henkel, "On the efficiency of voltage overscaling under temperature and aging effects," *IEEE Transactions on Computers*, vol. 68, no. 11, pp. 1647–1662, 2019.

[16] M. S. Mollison, J. P. Erickson, J. H. Anderson, S. K. Baruah, and J. A. Scoredos, "Mixed-Criticality Real-Time Scheduling for Multicore Systems," in *CIT*. IEEE, 2010, pp. 1864–1871.

[17] F. Zanini, D. Atienza, L. Benini, and G. De Micheli, "Multicore thermal management with model predictive control," in *2009 European Conference on Circuit Theory and Design*, 2009, pp. 711–714.

[18] E. F. Camacho and C. B. Alba, *Model predictive control*. Springer science & business media, 2013.

[19] A. Munshi, "The opencl specification," in *HCS*. IEEE, 2009, pp. 1–314.

[20] R. Huang and et al., "Yolo-lite: A real-time object detection algorithm optimized for non-gpu computers," in *IEEE BigData*, 2018.

[21] P. Sermanet and Y. LeCun, "Traffic sign recognition with multi-scale convolutional networks," in *The 2011 International Joint Conference on Neural Networks*. IEEE, 2011, pp. 2809–2813.

[22] M. Fan, V. Chaturvedi, S. Sha, and G. Quan, "An analytical solution for multi-core energy calculation with consideration of leakage and temperature dependency," in *International Symposium on Low Power Electronics and Design (ISLPED)*, 2013, pp. 353–358.

[23] M. Hochbruck and A. Ostermann, "Exponential integrators," *Acta Numerica*, vol. 19, pp. 209–286, 2010.

[24] S. Corbetta *et al.*, "Estimation of thermal status in multi-core systems," in *ISCAS*, 2011.

[25] C.-Y. Zhang, S. Luo, A. Huang, and C. Xu, "The eigenvalue distribution of block diagonally dominant matrices and block H-matrices," *The Electronic Journal of Linear Algebra*, vol. 20, pp. 621–639, 2010.

[26] B. Datta, *Numerical methods for linear control systems*. Academic Press, 2004, vol. 1.

[27] ALPriority. CAUTION: Smartphone operating temperatures. [Online]. Available: https://tinyurl.com/852djadn

[28] S. Ma, M. Jiang, P. Tao, C. Song, J. Wu, J. Wang, T. Deng, and W. Shang, "Temperature effect and thermal impact in lithium-ion batteries: A review," *Progress in Natural Science: Materials International*, vol. 28, no. 6, pp. 653–666, 2018.

[29] C. Imes, D. H. Kim, M. Maggio, and H. Hoffmann, "POET: A Portable Approach to Minimizing Energy under Soft Real-time Constraints," in *RTAS*. IEEE, 2015, pp. 75–86.

[30] N. Mishra, C. Imes, J. D. Lafferty, and H. Hoffmann, "CALOREE: Learning Control for Predictable Latency and Low Energy," in *ASPLOS*, 2018, pp. 184–198.

[31] "User Manual Odroid-XU4," https://magazine.odroid.com/wp-content/uploads/odroid-xu4-user-manual.pdf, [Online].

[32] P. Sermanet and Y. LeCun, "Traffic sign recognition with multi-scale Convolutional Networks," in *IJCNN*, 2011, pp. 2809–2813.

[33] K. Dev and S. Reda, "Scheduling Challenges and Opportunities in Integrated CPU+GPU Processors," in *ESTIMedia*. IEEE, 2016, p. 78–83.

[34] G. Singla, G. Kaur, A. K. Unver, and U. Y. Ogras, "Predictive Dynamic Thermal and Power Management for Heterogeneous Mobile Platforms," in *DATE*, 2015, pp. 960–965.

[35] A. Prakash, H. Amrouch, M. Shafique, T. Mitra, and J. Henkel, "Improving mobile gaming performance through cooperative cpu-gpu thermal management," in *DAC*. IEEE, 2016, pp. 1–6.

[36] S. Maity, A. Ghose, S. Dey, S. Park, and S. Chakrabarty, "Wip: Cooling by core-idling: Thermal-aware thread scheduling for mobile multicore processors," in *2021 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2021, pp. 520–523.

[37] Y. Lee, K. G. Shin, and H. S. Chwa, "Thermal-Aware Scheduling for Integrated CPUs-GPU Platforms," *ACM Transactions on Embedded Computing Systems*, vol. 18, no. 5s, p. 78–83, 2019.

[38] Q. Deng, D. Meisner, A. Bhattacharjee, T. F. Wenisch, and R. Bianchini, "Coscale: Coordinating CPU and memory system DVFS in server systems," in *MICRO*. IEEE, 2012, pp. 143–154.

[39] A. S. Dhodapkar and J. E. Smith, "Managing multi-configuration hardware via dynamic working set analysis," in *ISCA*. IEEE, 2002, pp. 233–244.

[40] S. Dey, E. Z. Guajardo, K. R. Basireddy, X. Wang, A. K. Singh, and K. McDonald-Maier, "EdgeCoolingMode: An Agent based Thermal Management Mechanism for DVFS enabled Heterogeneous MPSoCs," in *VLSID*. IEEE, 2019, pp. 19–24.

[41] C. Lu, J. A. Stankovic, S. H. Son, and G. Tao, "Feedback control real-time scheduling: Framework, modeling, and algorithms," *Real-Time Systems*, vol. 23, no. 1-2, pp. 85–126, 2002.

[42] A. M. Rahmani, B. Donyanavard, T. Mück, K. Moazzemi, A. Jantsch, O. Mutlu, and N. Dutt, "SPECTR: Formal Supervisory Control and Coordination for Many-core Systems Resource Management," in *ASPLOS*. ACM, 2018, pp. 169–183.

[43] J. L. Hellerstein, Y. Diao, S. Parekh, and D. M. Tilbury, *Feedback control of computing systems*. John Wiley & Sons, 2004.

[44] I. Yeo, C. C. Liu, and E. J. Kim, "Predictive dynamic thermal management for multicore systems," in *2008 45th ACM/IEEE Design Automation Conference*, 2008, pp. 734–739.

[45] B. Wojciechowski and J. Biernat, "Temperature prediction for multi-core microprocessors with application to Dynamic Thermal Management," in *18th International Workshop on THERMal INvestigation of ICs and Systems*, 2012, pp. 1–6.

[46] A. Bartolini, M. Cacciari, A. Tilli, and L. Benini, "A distributed and self-calibrating model-predictive controller for energy and thermal management of high-performance multicores," in *2011 Design, Automation Test in Europe*, 2011, pp. 1–6.

[47] A. Majumdar, L. Piga, I. Paul, J. L. Greathouse, W. Huang, and D. H. Albonesi, "Dynamic GPGPU Power Management Using Adaptive Model Predictive Control," in *HPCA*, 2017, pp. 613–624.

[48] S. Wang, G. Ananthanarayanan, and T. Mitra, "OPTiC: Optimizing collaborative CPU–GPU computing on mobile devices with thermal constraints," *IEEE transactions on computer-aided design of integrated circuits and systems*, vol. 38, no. 3, pp. 393–406, 2018.

[49] N. Stawinoga and T. Field, "Predictable thread coarsening," *TACO*, 2018.

[50] Y. Xing, S. Liang, L. Sui, X. Jia, J. Qiu, X. Liu, Y. Wang, Y. Shan, and Y. Wang, "DNNVM: End-to-end Compiler Leveraging Heterogeneous Optimizations on FPGA-based CNN Accelerators," *TCAD*, 2019.




Authorized licensed use limited to: Wuhan University. Downloaded on September 20,2026 at 11:58:48 UTC from IEEE Xplore. Restrictions apply.